Sequence-Level Knowledge Distillation

Yoon Kim
yoonkim@seas.harvard.edu

Alexander M. Rush
srush@seas.harvard.edu

School of Engineering and Applied Sciences
Harvard University
Cambridge, MA, USA

Abstract

Neural machine translation (NMT) offers a novel alternative formulation of translation that is potentially simpler than statistical approaches. However to reach competitive performance, NMT models need to be exceedingly large. In this paper we consider applying *knowledge distillation* approaches (Bucila et al., 2006; Hinton et al., 2015) that have proven successful for reducing the size of neural models in other domains to the problem of NMT. We demonstrate that standard knowledge distillation applied to word-level prediction can be effective for NMT, and also introduce two novel *sequence-level* versions of knowledge distillation that further improve performance, and somewhat surprisingly, seem to eliminate the need for beam search (even when applied on the original teacher model). Our best student model runs 10 times faster than its state-of-the-art teacher with little loss in performance. It is also significantly better than a baseline model trained without knowledge distillation: by $4.2/1.7$ BLEU with greedy decoding/beam search. Applying weight pruning on top of knowledge distillation results in a student model that has $13\times$ fewer parameters than the original teacher model, with a decrease of 0.4 BLEU.

1 Introduction

Neural machine translation (NMT) (Kalchbrenner and Blunsom, 2013; Cho et al., 2014; Sutskever et al., 2014; Bahdanau et al., 2015) is a deep learning-based method for translation that has recently shown promising results as an alternative to statistical approaches. NMT systems directly model the probability of the next word in the target sentence simply by conditioning a recurrent neural network on the source sentence and previously generated target words.

While both simple and surprisingly accurate, NMT systems typically need to have very high capacity in order to perform well: Sutskever et al. (2014) used a 4-layer LSTM with 1000 hidden units per layer (herein 4×1000) and Zhou et al. (2016) obtained state-of-the-art results on English $\rightarrow$ French with a 16-layer LSTM with 512 units per layer. The sheer size of the models requires cutting-edge hardware for training and makes using the models on standard setups very challenging.

This issue of excessively large networks has been observed in several other domains, with much focus on fully-connected and convolutional networks for multi-class classification. Researchers have particularly noted that large networks seem to be necessary for training, but learn redundant representations in the process (Denil et al., 2013). Therefore compressing deep models into smaller networks has been an active area of research. As deep learning systems obtain better results on NLP tasks, compression also becomes an important practical issue with applications such as running deep learning models for speech and translation locally on cell phones.

Existing compression methods generally fall into two categories: (1) *pruning* and (2) *knowledge distillation*. *Pruning* methods (LeCun et al., 1990; He et al., 2014; Han et al., 2016), zero-out weights or entire neurons based on an importance criterion: LeCun et al. (1990) use (a diagonal approximation to)

Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, pages 1317–1327,
Austin, Texas, November 1-5, 2016. ©2016 Association for Computational Linguistics

the Hessian to identify weights whose removal minimally impacts the objective function, while Han et al. (2016) remove weights based on thresholding their absolute values. *Knowledge distillation* approaches (Bucila et al., 2006; Ba and Caruana, 2014; Hinton et al., 2015) learn a smaller *student* network to mimic the original *teacher* network by minimizing the loss (typically L_2 or cross-entropy) between the student and teacher output.

In this work, we investigate knowledge distillation in the context of neural machine translation. We note that NMT differs from previous work which has mainly explored non-recurrent models in the multi-class prediction setting. For NMT, while the model is trained on multi-class prediction at the word-level, it is tasked with predicting complete sequence outputs conditioned on previous decisions. With this difference in mind, we experiment with standard knowledge distillation for NMT and also propose two new versions of the approach that attempt to approximately match the sequence-level (as opposed to word-level) distribution of the teacher network. This sequence-level approximation leads to a simple training procedure wherein the student network is trained on a newly generated dataset that is the result of running beam search with the teacher network.

We run experiments to compress a large state-of-the-art 4×1000 LSTM model, and find that with sequence-level knowledge distillation we are able to learn a 2×500 LSTM that roughly matches the performance of the full system. We see similar results compressing a 2×500 model down to 2×100 on a smaller data set. Furthermore, we observe that our proposed approach has other benefits, such as not requiring any beam search at test-time. As a result we are able to perform greedy decoding on the 2×500 model 10 times faster than beam search on the 4×1000 model with comparable performance. Our student models can even be run efficiently on a standard smartphone.[1] Finally, we apply weight pruning on top of the student network to obtain a model that has $13\times$ fewer parameters than the original teacher model. We have released all the code for the models described in this paper.[2]

[1] https://github.com/harvardnlp/nmt-android

[2] https://github.com/harvardnlp/seq2seq-attn

2 Background

2.1 Sequence-to-Sequence with Attention

Let $\mathbf{s} = [s_1, \ldots, s_I]$ and $\mathbf{t} = [t_1, \ldots, t_J]$ be (random variable sequences representing) the source/target sentence, with I and J respectively being the source/target lengths. Machine translation involves finding the most probable target sentence given the source:

$$\operatorname*{argmax}_{\mathbf{t} \in \mathcal{T}} p(\mathbf{t} \mid \mathbf{s})$$

where $\mathcal{T}$ is the set of all possible sequences. NMT models parameterize $p(\mathbf{t} \mid \mathbf{s})$ with an *encoder* neural network which reads the source sentence and a *decoder* neural network which produces a distribution over the target sentence (one word at a time) given the source. We employ the attentional architecture from Luong et al. (2015), which achieved state-of-the-art results on English $\rightarrow$ German translation.[3]

2.2 Knowledge Distillation

Knowledge distillation describes a class of methods for training a smaller *student* network to perform better by learning from a larger *teacher* network (in addition to learning from the training data set). We generally assume that the teacher has previously been trained, and that we are estimating parameters for the student. Knowledge distillation suggests training by matching the student's predictions to the teacher's predictions. For classification this usually means matching the probabilities either via L_2 on the $\log$ scale (Ba and Caruana, 2014) or by cross-entropy (Li et al., 2014; Hinton et al., 2015).

Concretely, assume we are learning a multi-class classifier over a data set of examples of the form (x, y) with possible classes $\mathcal{V}$. The usual training criteria is to minimize NLL for each example from the training data,

$$\mathcal{L}_{\text{NLL}}(\theta) = -\sum_{k=1}^{|\mathcal{V}|} \mathbb{1}\{y = k\} \log p(y = k \mid x; \theta)$$

where $\mathbb{1}\{\cdot\}$ is the indicator function and p the distribution from our model (parameterized by θ).

[3] Specifically, we use the *global-general* attention model with the *input-feeding* approach. We refer the reader to the original paper for further details.

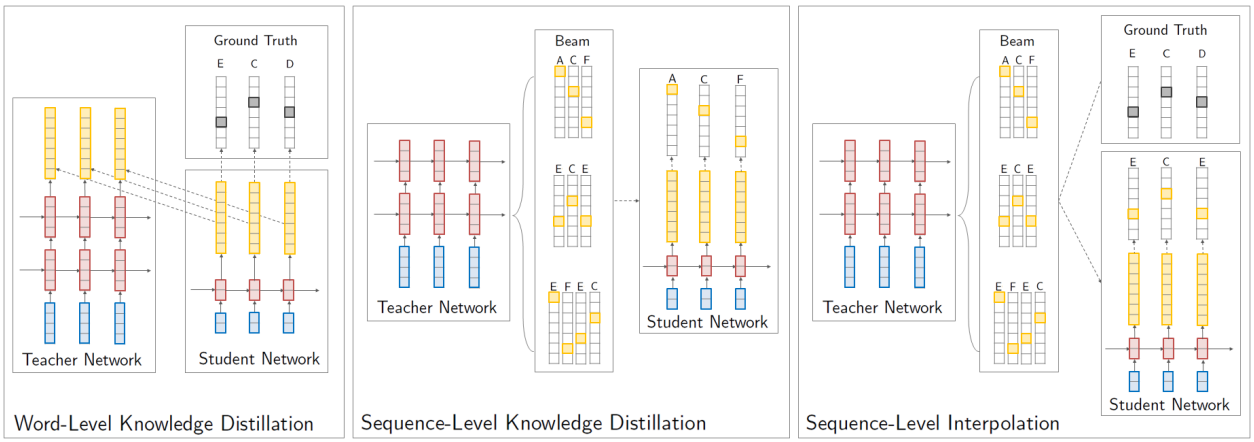

Figure 1: Overview of the different knowledge distillation approaches. In word-level knowledge distillation (left) cross-entropy is minimized between the student/teacher distributions (yellow) for each word in the actual target sequence (ECD), as well as between the student distribution and the degenerate data distribution, which has all of its probabilitiy mass on one word (black). In sequence-level knowledge distillation (center) the student network is trained on the output from beam search of the teacher network that had the highest score (ACF). In sequence-level interpolation (right) the student is trained on the output from beam search of the teacher network that had the highest sim with the target sequence (ECE).

This objective can be seen as minimizing the cross-entropy between the degenerate data distribution (which has all of its probability mass on one class) and the model distribution $p(y \mid x; \theta)$.

In knowledge distillation, we assume access to a learned teacher distribution $q(y \mid x; \theta_T)$, possibly trained over the same data set. Instead of minimizing cross-entropy with the observed data, we instead minimize the cross-entropy with the teacher's probability distribution,

$$\mathcal{L}_{\text{KD}}(\theta; \theta_T) = -\sum_{k=1}^{|\mathcal{V}|} q(y = k \mid x; \theta_T) \times \log p(y = k \mid x; \theta)$$

where θ_T parameterizes the teacher distribution and remains fixed. Note the cross-entropy setup is identical, but the target distribution is no longer a sparse distribution.[4] Training on $q(y \mid x; \theta_T)$ is attractive since it gives more information about other classes for a given data point (e.g. similarity between classes) and has less variance in gradients (Hinton et al., 2015).

[4] In some cases the entropy of the teacher/student distribution is increased by annealing it with a temperature term $\tau > 1$

$$\tilde{p}(y \mid x) \propto p(y \mid x)^{\frac{1}{\tau}}$$

After testing $\tau \in \{1, 1.5, 2\}$ we found that $\tau = 1$ worked best.

Since this new objective has no direct term for the training data, it is common practice to interpolate between the two losses,

$$\mathcal{L}(\theta; \theta_T) = (1 - \alpha)\mathcal{L}_{\text{NLL}}(\theta) + \alpha \mathcal{L}_{\text{KD}}(\theta; \theta_T)$$

where α is mixture parameter combining the one-hot distribution and the teacher distribution.

3 Knowledge Distillation for NMT

The large sizes of neural machine translation systems make them an ideal candidate for knowledge distillation approaches. In this section we explore three different ways this technique can be applied to NMT.

3.1 Word-Level Knowledge Distillation

NMT systems are trained directly to minimize word NLL, $\mathcal{L}_{\text{WORD-NLL}}$, at each position. Therefore if we have a teacher model, standard knowledge distillation for multi-class cross-entropy can be applied. We define this distillation for a sentence as,

$$\mathcal{L}_{\text{WORD-KD}} = -\sum_{j=1}^{J} \sum_{k=1}^{|\mathcal{V}|} q(t_j = k \mid \mathbf{s}, \mathbf{t}_{<j}) \times \log p(t_j = k \mid \mathbf{s}, \mathbf{t}_{<j})$$

where $\mathcal{V}$ is the target vocabulary set. The student can further be trained to optimize the mixture of

$\mathcal{L}_{\text{WORD-KD}}$ and $\mathcal{L}_{\text{WORD-NLL}}$. In the context of NMT, we refer to this approach as *word-level* knowledge distillation and illustrate this in Figure 1 (left).

3.2 Sequence-Level Knowledge Distillation

Word-level knowledge distillation allows transfer of these local word distributions. Ideally however, we would like the student model to mimic the teacher's actions at the *sequence-level*. The sequence distribution is particularly important for NMT, because wrong predictions can propagate forward at test-time.

First, consider the sequence-level distribution specified by the model over all possible sequences $\mathbf{t} \in \mathcal{T}$,

$$p(\mathbf{t} \,|\, \mathbf{s}) = \prod_{j=1}^{J} p(t_j \,|\, \mathbf{s}, \mathbf{t}_{<j})$$

for any length J. The sequence-level negative log-likelihood for NMT then involves matching the one-hot distribution over all complete sequences,

$$\begin{aligned} \mathcal{L}_{\text{SEQ-NLL}} &= -\sum_{\mathbf{t} \in \mathcal{T}} \mathbb{1}\{\mathbf{t} = \mathbf{y}\} \log p(\mathbf{t} \,|\, \mathbf{s}) \\ &= -\sum_{j=1}^{J} \sum_{k=1}^{|\mathcal{V}|} \mathbb{1}\{y_j = k\} \log p(t_j = k \,|\, \mathbf{s}, \mathbf{t}_{<j}) \\ &= \mathcal{L}_{\text{WORD-NLL}} \end{aligned}$$

where $\mathbf{y} = [y_1, \ldots, y_J]$ is the observed sequence. Of course, this just shows that from a negative log likelihood perspective, minimizing word-level NLL and sequence-level NLL are equivalent in this model.

But now consider the case of sequence-level knowledge distillation. As before, we can simply replace the distribution from the data with a probability distribution derived from our teacher model. However, instead of using a single word prediction, we use $q(\mathbf{t} \,|\, \mathbf{s})$ to represent the teacher's sequence distribution over the sample space of all possible sequences,

$$\mathcal{L}_{\text{SEQ-KD}} = -\sum_{\mathbf{t} \in \mathcal{T}} q(\mathbf{t} \,|\, \mathbf{s}) \log p(\mathbf{t} \,|\, \mathbf{s})$$

Note that $\mathcal{L}_{\text{SEQ-KD}}$ is inherently different from $\mathcal{L}_{\text{WORD-KD}}$, as the sum is over an exponential number of terms. Despite its intractability, we posit that this sequence-level objective is worthwhile. It gives the teacher the chance to assign probabilities to complete sequences and therefore transfer a broader range of knowledge. We thus consider an approximation of this objective.

Our simplest approximation is to replace the teacher distribution q with its mode,

$$q(\mathbf{t} \,|\, \mathbf{s}) \sim \mathbb{1}\{\mathbf{t} = \operatorname*{argmax}_{\mathbf{t} \in \mathcal{T}} q(\mathbf{t} \,|\, \mathbf{s})\}$$

Observing that finding the mode is itself intractable, we use beam search to find an approximation. The loss is then

$$\begin{aligned} \mathcal{L}_{\text{SEQ-KD}} &\approx -\sum_{\mathbf{t} \in \mathcal{T}} \mathbb{1}\{\mathbf{t} = \hat{\mathbf{y}}\} \log p(\mathbf{t} \,|\, \mathbf{s}) \\ &= -\log p(\mathbf{t} = \hat{\mathbf{y}} \,|\, \mathbf{s}) \end{aligned}$$

where $\hat{\mathbf{y}}$ is now the output from running beam search with the teacher model.

Using the mode seems like a poor approximation for the teacher distribution $q(\mathbf{t} \,|\, \mathbf{s})$, as we are approximating an exponentially-sized distribution with a single sample. However, previous results showing the effectiveness of beam search decoding for NMT lead us to belief that a large portion of q's mass lies in a single output sequence. In fact, in experiments we find that with beam of size 1, $q(\hat{\mathbf{y}} \,|\, \mathbf{s})$ (on average) accounts for 1.3% of the distribution for German $\rightarrow$ English, and 2.3% for Thai $\rightarrow$ English (Table 1: $p(\mathbf{t} = \hat{\mathbf{y}})$).[5]

To summarize, sequence-level knowledge distillation suggests to: (1) train a teacher model, (2) run beam search over the training set with this model, (3) train the student network with cross-entropy on this new dataset. Step (3) is identical to the word-level NLL process except now on the newly-generated data set. This is shown in Figure 1 (center).

[5] Additionally there are simple ways to better approximate $q(\mathbf{t} \,|\, \mathbf{s})$. One way would be to consider a K-best list from beam search and renormalizing the probabilities,

$$q(\mathbf{t} \,|\, \mathbf{s}) \sim \frac{q(\mathbf{t} \,|\, \mathbf{s})}{\sum_{\mathbf{t} \in \mathcal{T}_K} q(\mathbf{t} \,|\, \mathbf{s})}$$

where $\mathcal{T}_K$ is the K-best list from beam search. This would increase the training set by a factor of K. A beam of size 5 captures 2.8% of the distribution for German $\rightarrow$ English, and 3.8% for Thai $\rightarrow$ English. Another alternative is to use a Monte Carlo estimate and sample from the teacher model (since $\mathcal{L}_{\text{SEQ-KD}} = \mathbb{E}_{\mathbf{t} \sim q(\mathbf{t} \,|\, \mathbf{s})}[-\log p(\mathbf{t} \,|\, \mathbf{s})]$). However in practice we found the (approximate) mode to work well.

3.3 Sequence-Level Interpolation

Next we consider integrating the training data back into the process, such that we train the student model as a mixture of our sequence-level teacher-generated data ($\mathcal{L}_{\text{SEQ-KD}}$) with the original training data ($\mathcal{L}_{\text{SEQ-NLL}}$),

$$\begin{aligned}\mathcal{L} &= (1-\alpha)\mathcal{L}_{\text{SEQ-NLL}} + \alpha\mathcal{L}_{\text{SEQ-KD}} \\ &= -(1-\alpha)\log p(\mathbf{y}\,|\,\mathbf{s}) - \alpha\sum_{\mathbf{t}\in\mathcal{T}} q(\mathbf{t}\,|\,\mathbf{s})\log p(\mathbf{t}\,|\,\mathbf{s})\end{aligned}$$

where $\mathbf{y}$ is the gold target sequence.

Since the second term is intractable, we could again apply the mode approximation from the previous section,

$$\mathcal{L} = -(1-\alpha)\log p(\mathbf{y}\,|\,\mathbf{s}) - \alpha\log p(\hat{\mathbf{y}}\,|\,\mathbf{s})$$

and train on both observed ($\mathbf{y}$) and teacher-generated ($\hat{\mathbf{y}}$) data. However, this process is non-ideal for two reasons: (1) unlike for standard knowledge distribution, it doubles the size of the training data, and (2) it requires training on both the teacher-generated sequence and the true sequence, conditioned on the same source input. The latter concern is particularly problematic since we observe that $\mathbf{y}$ and $\hat{\mathbf{y}}$ are often quite different.

As an alternative, we propose a single-sequence approximation that is more attractive in this setting. This approach is inspired by *local updating* (Liang et al., 2006), a method for discriminative training in statistical machine translation (although to our knowledge not for knowledge distillation). Local updating suggests selecting a training sequence which is close to $\mathbf{y}$ *and* has high probability under the teacher model,

$$\tilde{\mathbf{y}} = \underset{\mathbf{t}\in\mathcal{T}}{\operatorname{argmax}}\, sim(\mathbf{t},\mathbf{y})q(\mathbf{t}\,|\,\mathbf{s})$$

where sim is a function measuring closeness (e.g. Jaccard similarity or BLEU (Papineni et al., 2002)). Following local updating, we can approximate this sequence by running beam search and choosing

$$\tilde{\mathbf{y}} \approx \underset{\mathbf{t}\in\mathcal{T}_K}{\operatorname{argmax}}\, sim(\mathbf{t},\mathbf{y})$$

where $\mathcal{T}_K$ is the K-best list from beam search. We take sim to be smoothed sentence-level BLEU (Chen and Cherry, 2014).

We justify training on $\tilde{\mathbf{y}}$ from a knowledge distillation perspective with the following generative process: suppose that there is a true target sequence (which we do not observe) that is first generated from the underlying data distribution $\mathcal{D}$. And further suppose that the target sequence that we observe ($\mathbf{y}$) is a noisy version of the unobserved true sequence: i.e. (i) $\mathbf{t} \sim \mathcal{D}$, (ii) $\mathbf{y} \sim \epsilon(\mathbf{t})$, where $\epsilon(\mathbf{t})$ is, for example, a noise function that independently replaces each element in $\mathbf{t}$ with a random element in $\mathcal{V}$ with some small probability.[6] In such a case, ideally the student's distribution should match the mixture distribution,

$$\mathcal{D}_{\text{SEQ-Inter}} \sim (1-\alpha)\mathcal{D} + \alpha q(\mathbf{t}\,|\,\mathbf{s})$$

In this setting, due to the noise assumption, $\mathcal{D}$ now has significant probability mass around a neighborhood of $\mathbf{y}$ (not just at $\mathbf{y}$), and therefore the argmax of the mixture distribution is likely something other than $\mathbf{y}$ (the observed sequence) or $\hat{\mathbf{y}}$ (the output from beam search). We can see that $\tilde{\mathbf{y}}$ is a natural approximation to the argmax of this mixture distribution between $\mathcal{D}$ and $q(\mathbf{t}\,|\,\mathbf{s})$ for some α. We illustrate this framework in Figure 1 (right) and visualize the distribution over a real example in Figure 2.

4 Experimental Setup

To test out these approaches, we conduct two sets of NMT experiments: high resource (English $\rightarrow$ German) and low resource (Thai $\rightarrow$ English).

The **English-German** data comes from WMT 2014.[7] The training set has 4m sentences and we take newstest2012/newstest2013 as the dev set and newstest2014 as the test set. We keep the top 50k most frequent words, and replace the rest with UNK. The teacher model is a 4×1000 LSTM (as in Luong et al. (2015)) and we train two student models: 2×300 and 2×500. The **Thai-English** data comes from IWSLT 2015.[8] There are 90k sentences in the

[6] While we employ a simple (unrealistic) noise function for illustrative purposes, the generative story is quite plausible if we consider a more elaborate noise function which includes additional sources of noise such as phrase reordering, replacement of words with synonyms, etc. One could view translation having two sources of variance that should be modeled separately: variance due to the source sentence ($\mathbf{t} \sim \mathcal{D}$), and variance due to the individual translator ($\mathbf{y} \sim \epsilon(\mathbf{t})$).

[7] http://statmt.org/wmt14

[8] https://sites.google.com/site/iwsltevaluation2015/mt-track

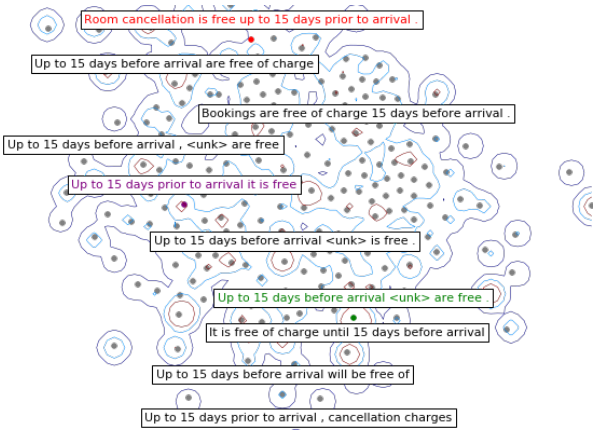

Figure 2: Visualization of sequence-level interpolation on an example German → English sentence: Bis 15 Tage vor Anreise sind Zimmer-Annullationen kostenlos. We run beam search, plot the final hidden state of the hypotheses using t-SNE and show the corresponding (smoothed) probabilities with contours. In the above example, the sentence that is at the top of the beam after beam search (green) is quite far away from gold (red), so we train the model on a sentence that is on the beam but had the highest sim (e.g. BLEU) to gold (purple).

training set and we take 2010/2011/2012 data as the dev set and 2012/2013 as the test set, with a vocabulary size is 25k. Size of the teacher model is 2×500 (which performed better than 4×1000, 2×750 models), and the student model is 2×100. Other training details mirror Luong et al. (2015).

We evaluate on tokenized BLEU with `multi-bleu.perl`, and experiment with the following variations:

Word-Level Knowledge Distillation (Word-KD) Student is trained on the original data and additionally trained to minimize the cross-entropy of the teacher distribution at the word-level. We tested $\alpha \in \{0.5, 0.9\}$ and found $\alpha = 0.5$ to work better.

Sequence-Level Knowledge Distillation (Seq-KD) Student is trained on the teacher-generated data, which is the result of running beam search and taking the highest-scoring sequence with the teacher model. We use beam size $K = 5$ (we did not see improvements with a larger beam).

Sequence-Level Interpolation (Seq-Inter) Student is trained on the sequence on the teacher's beam that had the highest BLEU (beam size $K = 35$). We adopt a fine-tuning approach where we begin training from a pretrained model (either on original data or Seq-KD data) and train with a smaller learning rate (0.1). For English-German we generate Seq-Inter data on a smaller portion of the training set ($\sim 50\%$) for efficiency.

The above methods are complementary and can be combined with each other. For example, we can train on teacher-generated data but still include a word-level cross-entropy term between the teacher/student (Seq-KD + Word-KD in Table 1), or fine-tune towards Seq-Inter data starting from the baseline model trained on original data (Baseline + Seq-Inter in Table 1).[9]

5 Results and Discussion

Results of our experiments are shown in Table 1. We find that while word-level knowledge distillation (Word-KD) does improve upon the baseline, sequence-level knowledge distillation (Seq-KD) does better on English → German and performs similarly on Thai → English. Combining them (Seq-KD + Word-KD) results in further gains for the 2×300 and 2×100 models (although not for the 2×500 model), indicating that these methods provide orthogonal means of transferring knowledge from the teacher to the student: Word-KD is transferring knowledge at the the local (i.e. word) level while Seq-KD is transferring knowledge at the global (i.e. sequence) level.

Sequence-level interpolation (Seq-Inter), in addition to improving models trained via Word-KD and Seq-KD, also improves upon the original teacher model that was trained on the actual data but fine-tuned towards Seq-Inter data (Baseline + Seq-Inter). In fact, greedy decoding with this fine-tuned model has similar performance (19.6) as beam search with the original model (19.5), allowing for faster decoding even with an identically-sized model.

We hypothesize that sequence-level knowledge distillation is effective because it allows the student network to only model relevant parts of the teacher distribution (i.e. around the teacher's mode) instead of 'wasting' parameters on trying to model the entire

[9]For instance, 'Seq-KD + Seq-Inter + Word-KD' in Table 1 means that the model was trained on Seq-KD data and fine-tuned towards Seq-Inter data with the mixture cross-entropy loss at the word-level.

Model	$\text{BLEU}_{K=1}$	$\Delta_{K=1}$	$\text{BLEU}_{K=5}$	$\Delta_{K=5}$	PPL	$p(\mathbf{t} = \hat{\mathbf{y}})$
English → German WMT 2014						
Teacher Baseline 4×1000 (Params: 221m)	17.7	–	19.5	–	6.7	1.3%
Baseline + Seq-Inter	19.6	+1.9	19.8	+0.3	10.4	8.2%
Student Baseline 2×500 (Params: 84m)	14.7	–	17.6	–	8.2	0.9%
Word-KD	15.4	+0.7	17.7	+0.1	8.0	1.0%
Seq-KD	18.9	**+4.2**	19.0	+1.4	22.7	16.9%
Baseline + Seq-Inter	18.5	+3.6	18.7	+1.1	11.3	5.7%
Word-KD + Seq-Inter	18.3	+3.6	18.5	+0.9	11.8	6.3%
Seq-KD + Seq-Inter	18.9	**+4.2**	19.3	**+1.7**	15.8	7.6%
Seq-KD + Word-KD	18.7	+4.0	18.9	+1.3	10.9	4.1%
Seq-KD + Seq-Inter + Word-KD	18.8	+4.1	19.2	+1.6	14.8	7.1%
Student Baseline 2×300 (Params: 49m)	14.1	–	16.9	–	10.3	0.6%
Word-KD	14.9	+0.8	17.6	+0.7	10.9	0.7%
Seq-KD	18.1	+4.0	18.1	+1.2	64.4	14.8%
Baseline + Seq-Inter	17.6	+3.5	17.9	+1.0	13.0	10.0%
Word-KD + Seq-Inter	17.8	+3.7	18.0	+1.1	14.5	4.3%
Seq-KD + Seq-Inter	18.2	+4.1	18.5	+1.6	40.8	5.6%
Seq-KD + Word-KD	17.9	+3.8	18.8	+1.9	44.1	3.1%
Seq-KD + Seq-Inter + Word-KD	18.5	**+4.4**	18.9	**+2.0**	97.1	5.9%
Thai → English IWSLT 2015						
Teacher Baseline 2×500 (Params: 47m)	14.3	–	15.7	–	22.9	2.3%
Baseline + Seq-Inter	15.6	+1.3	16.0	+0.3	55.1	6.8%
Student Baseline 2×100 (Params: 8m)	10.6	–	12.7	–	37.0	1.4%
Word-KD	11.8	+1.2	13.6	+0.9	35.3	1.4%
Seq-KD	12.8	+2.2	13.4	+0.7	125.4	6.9%
Baseline + Seq-Inter	12.9	+2.3	13.1	+0.4	52.8	2.5%
Word-KD + Seq-Inter	13.0	+2.4	13.7	+1.0	58.7	3.2%
Seq-KD + Seq-Inter	13.6	+3.0	14.0	+1.3	106.4	3.9%
Seq-KD + Word-KD	13.7	+3.1	14.2	+1.5	67.4	3.1%
Seq-KD + Seq-Inter + Word-KD	14.2	**+3.6**	14.4	**+1.7**	117.4	3.2%

Table 1: Results on English-German (newstest2014) and Thai-English (2012/2013) test sets. $\text{BLEU}_{K=1}$: BLEU score with beam size $K = 1$ (i.e. greedy decoding); $\Delta_{K=1}$: BLEU gain over the baseline model without any knowledge distillation with greedy decoding; $\text{BLEU}_{K=5}$: BLEU score with beam size $K = 5$; $\Delta_{K=5}$: BLEU gain over the baseline model without any knowledge distillation with beam size $K = 5$; PPL: perplexity on the test set; $p(\mathbf{t} = \hat{\mathbf{y}})$: Probability of output sequence from greedy decoding (averaged over the test set). Params: number of parameters in the model. Best results (as measured by improvement over the baseline) within each category are highlighted in bold.

space of translations. Our results suggest that this is indeed the case: the probability mass that Seq-KD models assign to the approximate mode is much higher than is the case for baseline models trained on original data (Table 1: $p(\mathbf{t} = \hat{\mathbf{y}})$). For example, on English → German the (approximate) argmax for the 2×500 Seq-KD model (on average) accounts for 16.9% of the total probability mass, while the corresponding number is 0.9% for the baseline. This also explains the success of greedy decoding for Seq-KD models—since we are only modeling around the teacher's mode, the student's distribution is more peaked and therefore the argmax is much easier to find. Seq-Inter offers a compromise between the two, with the greedily-decoded sequence accounting for 7.6% of the distribution.

Finally, although past work has shown that models with lower perplexity generally tend to have

Model Size	GPU	CPU	Android
Beam = 1 (Greedy)			
4×1000	425.5	15.0	–
2×500	1051.3	63.6	8.8
2×300	1267.8	104.3	15.8
Beam $= 5$			
4×1000	101.9	7.9	–
2×500	181.9	22.1	1.9
2×300	189.1	38.4	3.4

Table 2: Number of source words translated per second across GPU (GeForce GTX Titan X), CPU, and smartphone (Samsung Galaxy 6) for the various English $\rightarrow$ German models. We were unable to open the 4×1000 model on the smartphone.

Model	Prune %	Params	BLEU	Ratio
4×1000	0%	221 m	19.5	$1\times$
2×500	0%	84 m	19.3	$3\times$
2×500	50%	42 m	19.3	$5\times$
2×500	80%	17 m	19.1	$13\times$
2×500	85%	13 m	18.8	$18\times$
2×500	90%	8 m	18.5	$26\times$

Table 3: Performance of student models with varying % of the weights pruned. Top two rows are models without any pruning. Params: number of parameters in the model; Prune %: Percentage of weights pruned based on their absolute values; BLEU: BLEU score with beam search decoding ($K = 5$) after retraining the pruned model; Ratio: Ratio of the number of parameters versus the original teacher model (which has 221m parameters).

higher BLEU, our results indicate that this is not necessarily the case. The perplexity of the baseline 2×500 English $\rightarrow$ German model is 8.2 while the perplexity of the corresponding Seq-KD model is 22.7, despite the fact that Seq-KD model does significantly better for both greedy (+4.2 BLEU) and beam search (+1.4 BLEU) decoding.

5.1 Decoding Speed

Run-time complexity for beam search grows linearly with beam size. Therefore, the fact that sequence-level knowledge distillation allows for greedy decoding is significant, with practical implications for running NMT systems across various devices. To test the speed gains, we run the teacher/student models on GPU, CPU, and smartphone, and check the average number of source words translated per second (Table 2). We use a GeForce GTX Titan X for GPU and a Samsung Galaxy 6 smartphone. We find that we can run the student model 10 times faster with greedy decoding than the teacher model with beam search on GPU (1051.3 vs 101.9 words/sec), with similar performance.

5.2 Weight Pruning

Although knowledge distillation enables training faster models, the number of parameters for the student models is still somewhat large (Table 1: Params), due to the word embeddings which dominate most of the parameters.[10] For example, on the 2×500 English $\rightarrow$ German model the word embeddings account for approximately 63% (50m out of 84m) of the parameters. The size of word embeddings have little impact on run-time as the word embedding layer is a simple lookup table that only affects the first layer of the model.

We therefore focus next on reducing the memory footprint of the student models further through weight pruning. Weight pruning for NMT was recently investigated by See et al. (2016), who found that up to $80 - 90\%$ of the parameters in a large NMT model can be pruned with little loss in performance. We take our best English $\rightarrow$ German student model (2×500 Seq-KD + Seq-Inter) and prune $x\%$ of the parameters by removing the weights with the lowest absolute values. We then retrain the pruned model on Seq-KD data with a learning rate of 0.2 and fine-tune towards Seq-Inter data with a learning rate of 0.1. As observed by See et al. (2016), retraining proved to be crucial. The results are shown in Table 3.

Our findings suggest that compression benefits achieved through weight pruning and knowledge distillation are orthogonal.[11] Pruning 80% of the weight in the 2×500 student model results in a model with $13\times$ fewer parameters than the original teacher model with only a decrease of 0.4 BLEU. While pruning 90% of the weights results in a more appreciable decrease of 1.0 BLEU, the model is

[10] Word embeddings scale linearly while RNN parameters scale quadratically with the dimension size.

[11] To our knowledge combining pruning and knowledge distillation has not been investigated before.

drastically smaller with 8m parameters, which is $26\times$ fewer than the original teacher model.

5.3 Further Observations

- For models trained with word-level knowledge distillation, we also tried regressing the student network's top-most hidden layer at each time step to the teacher network's top-most hidden layer as a pretraining step, noting that Romero et al. (2015) obtained improvements with a similar technique on feed-forward models. We found this to give comparable results to standard knowledge distillation and hence did not pursue this further.

- There have been promising recent results on eliminating word embeddings completely and obtaining word representations directly from characters with character composition models, which have many fewer parameters than word embedding lookup tables (Ling et al., 2015a; Kim et al., 2016; Ling et al., 2015b; Jozefowicz et al., 2016; Costa-Jussa and Fonollosa, 2016). Combining such methods with knowledge distillation/pruning to further reduce the memory footprint of NMT systems remains an avenue for future work.

6 Related Work

Compressing deep learning models is an active area of current research. *Pruning* methods involve pruning weights or entire neurons/nodes based on some criterion. LeCun et al. (1990) prune weights based on an approximation of the Hessian, while Han et al. (2016) show that a simple magnitude-based pruning works well. Prior work on removing neurons/nodes include Srinivas and Babu (2015) and Mariet and Sra (2016). See et al. (2016) were the first to apply pruning to Neural Machine Translation, observing that that different parts of the architecture (input word embeddings, LSTM matrices, etc.) admit different levels of pruning. *Knowledge distillation* approaches train a smaller student model to mimic a larger teacher model, by minimizing the loss between the teacher/student predictions (Bucila et al., 2006; Ba and Caruana, 2014; Li et al., 2014; Hinton et al., 2015). Romero et al. (2015) additionally regress on the intermediate hidden layers of the student/teacher network as a pretraining step, while Mou et al. (2015) obtain smaller word embeddings from a teacher model via regression. There has also been work on transferring knowledge across different network architectures: Chan et al. (2015b) show that a deep non-recurrent neural network can learn from an RNN; Geras et al. (2016) train a CNN to mimic an LSTM for speech recognition. Kuncoro et al. (2016) recently investigated knowledge distillation for structured prediction by having a single parser learn from an ensemble of parsers.

Other approaches for compression involve low rank factorizations of weight matrices (Denton et al., 2014; Jaderberg et al., 2014; Lu et al., 2016; Prabhavalkar et al., 2016), sparsity-inducing regularizers (Murray and Chiang, 2015), binarization of weights (Courbariaux et al., 2016; Lin et al., 2016), and weight sharing (Chen et al., 2015; Han et al., 2016). Finally, although we have motivated sequence-level knowledge distillation in the context of training a smaller model, there are other techniques that train on a mixture of the model's predictions and the data, such as local updating (Liang et al., 2006), hope/fear training (Chiang, 2012), SEARN (Daumé III et al., 2009), DAgger (Ross et al., 2011), and minimum risk training (Och, 2003; Shen et al., 2016).

7 Conclusion

In this work we have investigated existing knowledge distillation methods for NMT (which work at the word-level) and introduced two *sequence-level* variants of knowledge distillation, which provide improvements over standard word-level knowledge distillation.

We have chosen to focus on translation as this domain has generally required the largest capacity deep learning models, but the sequence-to-sequence framework has been successfully applied to a wide range of tasks including parsing (Vinyals et al., 2015a), summarization (Rush et al., 2015), dialogue (Vinyals and Le, 2015; Serban et al., 2016; Li et al., 2016), NER/POS-tagging (Gillick et al., 2016), image captioning (Vinyals et al., 2015b; Xu et al., 2015), video generation (Srivastava et al., 2015), and speech recognition (Chan et al., 2015a). We anticipate that methods described in this paper can be used to similarly train smaller models in other domains.

References

[Ba and Caruana2014] Lei Jimmy Ba and Rich Caruana. 2014. Do Deep Nets Really Need to be Deep? In *Proceedings of NIPS*.

[Bahdanau et al.2015] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2015. Neural Machine Translation by Jointly Learning to Align and Translate. In *Proceedings of ICLR*.

[Bucila et al.2006] Cristian Bucila, Rich Caruana, and Alexandru Niculescu-Mizil. 2006. Model Compression. In *Proceedings of KDD*.

[Chan et al.2015a] William Chan, Navdeep Jaitly, Quoc Le, and Oriol Vinyals. 2015a. Listen, Attend and Spell. *arXiv:1508.01211*.

[Chan et al.2015b] William Chan, Nan Rosemary Ke, and Ian Laner. 2015b. Transfering Knowledge from a RNN to a DNN. *arXiv:1504.01483*.

[Chen and Cherry2014] Boxing Chen and Colin Cherry. 2014. A Systematic Comparison of Smoothing Techniques for Sentence-Level BLEU. In *Proceedings of the Ninth Workshop on Statistical Machine Translation*.

[Chen et al.2015] Wenlin Chen, James T. Wilson, Stephen Tyree, Kilian Q. Weinberger, and Yixin Chen. 2015. Compressing Neural Networks with the Hashing Trick. In *Proceedings of ICML*.

[Chiang2012] David Chiang. 2012. Hope and Fear for Discriminative Training of Statistical Translation Models. In *JMLR*.

[Cho et al.2014] Kyunghyun Cho, Bart van Merrienboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. 2014. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *Proceedings of EMNLP*.

[Costa-Jussa and Fonollosa2016] Marta R. Costa-Jussa and Jose A.R. Fonollosa. 2016. Character-based Neural Machine Translation. *arXiv:1603.00810*.

[Courbariaux et al.2016] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. 2016. Binarized Neural Networks: Training Neural Networks with Weights and Activations Constrained to $+1$ or -1. *arXiv:1602.02830*.

[Daumé III et al.2009] Hal Daumé III, John Langford, and Daniel Marcu. 2009. Search-based Structured Prediction. *Machine Learning*.

[Denil et al.2013] Misha Denil, Babak Shakibi, Laurent Dinh, Marc'Aurelio Ranzato, and Nando de Freitas. 2013. Predicting Parameters in Deep Learning. In *Proceedings of NIPS*.

[Denton et al.2014] Emily L. Denton, Wojciech Zaremba, Joan Bruna, Yann LeCun, and Rob Fergus. 2014. Exploiting Linear Structure within Convolutional Neural Networks for Efficient Evaluation. In *Proceedings of NIPS*.

[Geras et al.2016] Krzysztof J. Geras, Abdel rahman Mohamed, Rich Caruana, Gregor Urban, Shengjie Wang, Ozlem Aslan, Matthai Philipose, Matthew Richardson, and Charles Sutton. 2016. Blending LSTMs into CNNs. In *Proceedings of ICLR Workshop*.

[Gillick et al.2016] Dan Gillick, Cliff Brunk, Oriol Vinyals, and Amarnag Subramanya. 2016. Multilingual Language Processing from Bytes. In *Proceedings of NAACL*.

[Han et al.2016] Song Han, Huizi Mao, and William J. Dally. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *Proceedings of ICLR*.

[He et al.2014] Tianxing He, Yuchen Fan, Yanmin Qian, Tian Tan, and Kai Yu. 2014. Reshaping Deep Neural Network for Fast Decoding by Node-Pruning. In *Proceedings of ICASSP*.

[Hinton et al.2015] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. *arXiv:1503.0253*.

[Jaderberg et al.2014] Max Jaderberg, Andrea Vedaldi, and Andrew Zisserman. 2014. Speeding up Convolutional Neural Networks with Low Rank Expansions. In *BMCV*.

[Jozefowicz et al.2016] Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer, and Yonghui Wu. 2016. Exploring the Limits of Language Modeling. *arXiv:1602.02410*.

[Kalchbrenner and Blunsom2013] Nal Kalchbrenner and Phil Blunsom. 2013. Recurrent Continuous Translation Models. In *Proceedings of EMNLP*.

[Kim et al.2016] Yoon Kim, Yacine Jernite, David Sontag, and Alexander M. Rush. 2016. Character-Aware Neural Language Models. In *Proceedings of AAAI*.

[Kuncoro et al.2016] Adhiguna Kuncoro, Miguel Ballesteros, Lingpeng Kong, Chris Dyer, and Noah A. Smith. 2016. Distilling an Ensemble of Greedy Dependency Parsers into One MST Parser. In *Proceedings of EMNLP*.

[LeCun et al.1990] Yann LeCun, John S. Denker, and Sara A. Solla. 1990. Optimal Brain Damage. In *Proceedings of NIPS*.

[Li et al.2014] Jinyu Li, Rui Zhao, Jui-Ting Huang, and Yifan Gong. 2014. Learning Small-Size DNN with Output-Distribution-Based Criteria. In *Proceedings of INTERSPEECH*.

[Li et al.2016] Jiwei Li, Michael Galley, Chris Brockett, Jianfeg Gao, and Bill Dolan. 2016. A Diversity-Promoting Objective Function for Neural Conversational Models. In *Proceedings of NAACL 2016*.

[Liang et al.2006] Percy Liang, Alexandre Bouchard-Cote, Dan Klein, and Ben Taskar. 2006. An End-to-End Discriminative Approach to Machine Translation. In *Proceedings of COLING-ACL*.

[Lin et al.2016] Zhouhan Lin, Matthieu Coubariaux, Roland Memisevic, and Yoshua Bengio. 2016. Neural Networks with Few Multiplications. In *Proceedings of ICLR*.

[Ling et al.2015a] Wang Ling, Tiago Lui, Luis Marujo, Ramon Fernandez Astudillo, Silvio Amir, Chris Dyer, Alan W Black, and Isabel Trancoso. 2015a. Finding Function in Form: Composition Character Models for Open Vocabulary Word Representation. In *Proceedings of EMNLP*.

[Ling et al.2015b] Wang Ling, Isabel Trancoso, Chris Dyer, and Alan W Black. 2015b. Character-based Neural Machine Translation. *arXiv:1511.04586*.

[Lu et al.2016] Zhiyun Lu, Vikas Sindhwani, and Tara N. Sainath. 2016. Learning Compact Recurrent Neural Networks. In *Proceedings of ICASSP*.

[Luong et al.2015] Minh-Thang Luong, Hieu Pham, and Christopher D. Manning. 2015. Effective Approaches to Attention-based Neural Machine Translation. In *Proceedings of EMNLP*.

[Mariet and Sra2016] Zelda Mariet and Suvrit Sra. 2016. Diversity Networks. In *Proceedings of ICLR*.

[Mou et al.2015] Lili Mou, Ge Li, Yan Xu, Lu Zhang, and Zhi Jin. 2015. Distilling Word Embeddings: An Encoding Approach. *arXiv:1506.04488*.

[Murray and Chiang2015] Kenton Murray and David Chiang. 2015. Auto-sizing Neural Networks: With Applications to N-Gram Language Models. In *Proceedings of EMNLP*.

[Och2003] Franz J. Och. 2003. Minimum Error Rate Training in Statistical Machine Translation. In *Proceedings of ACL*.

[Papineni et al.2002] Kishore Papineni, Slim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A Method for Automatic Evaluation of Machine Translation. In *Proceedings of ICML*.

[Prabhavalkar et al.2016] Rohit Prabhavalkar, Ouais Alsharif, Antoine Bruguier, and Ian McGraw. 2016. On the Compression of Recurrent Neural Networks with an Application to LVCSR Acoustic Modeling for Embedded Speech Recognition. In *Proceedings of ICASSP*.

[Romero et al.2015] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. 2015. FitNets: Hints for Thin Deep Nets. In *Proceedings of ICLR*.

[Ross et al.2011] Stephane Ross, Geoffrey J. Gordon, and Drew Bagnell. 2011. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *Proceedings of AISTATS*.

[Rush et al.2015] Alexander M. Rush, Sumit Chopra, and Jason Weston. 2015. A Neural Attention Model for Abstractive Sentence Summarization. In *Proceedings of EMNLP*.

[See et al.2016] Abigail See, Minh-Thang Luong, and Christopher D. Manning. 2016. Compression of Neural Machine Translation via Pruning. In *Proceedings of CoNLL*.

[Serban et al.2016] Iulian V. Serban, Allesandro Sordoni, Yoshua Bengio, Aaron Courville, and Joelle Pineau. 2016. Building End-to-End Dialogue Systems Using Generative Hierarchical Neural Network Models. In *Proceedings of AAAI*.

[Shen et al.2016] Shiqi Shen, Yong Cheng, Zhongjun He, Wei He, Hua Wu, Masong Sun, and Yang Liu. 2016. Minimum Risk Training for Neural Machine Translation. In *Proceedings of ACL*.

[Srinivas and Babu2015] Suraj Srinivas and R. Venkatesh Babu. 2015. Data-free Parameter Pruning for Deep Neural Networks. *BMVC*.

[Srivastava et al.2015] Nitish Srivastava, Elman Mansimov, and Ruslan Salakhutdinov. 2015. Unsupervised Learning of Video Representations using LSTMs. *Proceedings of ICML*.

[Sutskever et al.2014] Ilya Sutskever, Oriol Vinyals, and Quoc Le. 2014. Sequence to Sequence Learning with Neural Networks. In *Proceedings of NIPS*.

[Vinyals and Le2015] Oriol Vinyals and Quoc Le. 2015. A Neural Conversational Model. In *Proceedings of ICML Deep Learning Workshop*.

[Vinyals et al.2015a] Oriol Vinyals, Lukasz Kaiser, Terry Koo, Slave Petrov, Ilya Sutskever, and Geoffrey Hinton. 2015a. Grammar as a Foreign Language. In *Proceedings of NIPS*.

[Vinyals et al.2015b] Oriol Vinyals, Alexander Toshev, Samy Bengio, and Dumitru Erhan. 2015b. Show and Tell: A Neural Image Caption Generator. In *Proceedings of CVPR*.

[Xu et al.2015] Kelvin Xu, Jimma Ba, Ryan Kiros, Kyunghyun Cho, Aaron Courville, Ruslan Salakhutdinov, Richard Zemel, and Yoshua Bengio. 2015. Show, Attend and Tell: Neural Image Caption Generation with Visual Attention. In *Proceedings of ICML*.

[Zhou et al.2016] Jie Zhou, Ying Cao, Xuguang Wang, Peng Li, and Wei Xu. 2016. Deep Recurrent Models with Fast-Forward Connections for Neural Machine Translation. In *Proceedings of TACL*.